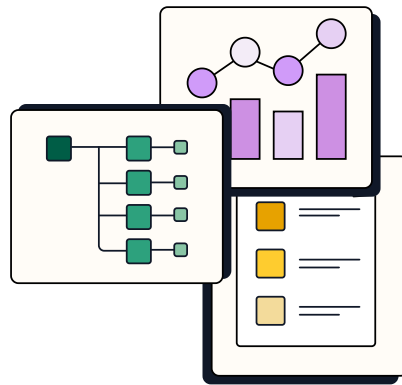


Certified Cool! groundcover has been recognized as a 2025 Gartner® Cool Vendor in Container Management



Home > **Microservices Observability**



Microservices Observability: Patterns, Pillars & Tools

Discover the essentials of Microservices Observability, What is it, key pillars like logs and metrics & patterns for effective cloud app management.

If you build or manage software today, it's very likely that working with microservices is part of your daily routine. Not all applications use a [microservices architecture](#), but in today's cloud-native world,

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

By the same token, microservices observability has become an essential component of monitoring and observability strategies for most teams. If you want to keep your microservices applications running as they should, you need a plan in place for handling the unique challenges of observing, troubleshooting and optimizing the performance of microservices.

What is Microservices Observability?

Microservices observability is the practice of achieving visibility into applications composed of microservices.

To understand fully what that means, let's unpack the definition of microservices. Microservices are an architectural strategy that breaks applications into a set of individual services – as opposed to operating as a monolith, which is the way applications traditionally ran.

Because microservices applications consist of multiple

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

service instance. At the same time, however, observability for microservices also hinges on correlating observability data from multiple services to gain insight into the health of the application as a whole.

Observability vs. Monitoring

It's important to note that observability is not just another word for using a monitoring tool. On the contrary, whereas monitoring refers to the collection and display of data, observability goes further by interpreting what data reveals about the internal state of an application.

Thus, while monitoring is often a step toward observability, monitoring alone does not enable observability. Observability only happens when you can take the data you collect through monitoring and interpret it in ways that deliver deep visibility into the state of your application – as opposed merely to knowing how the application appears from the outside.

Read more about
[Microservices Monitoring.](#)

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

Observability Important?



Observability is important for managing the health and performance of any type of software. However, in the context of microservices, observability is especially critical.

The main reason why is that by their nature, microservices applications are complex. They consist of multiple components that are continuously interacting with each other in complicated ways. A problem in one microservice (such as a microservice that powers an application's frontend) may impact the performance of others (like one that retrieves data from a backend database) in ways that are not always obvious through surface-level monitoring alone.

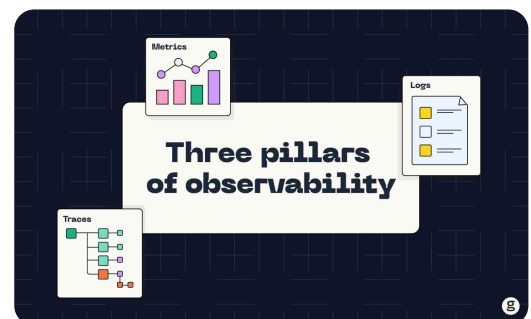
With observability, teams gain insight into the complex interactions that occur within

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

In addition, microservices observability helps to optimize resource utilization and reduce costs. When you understand what your apps are doing and can quickly determine the root cause of performance anomalies, you'll know whether your applications simply require more resources (which will in turn lead to increased infrastructure expenses), or if that approach would simply waste money without fixing underlying issues.

Key Pillars of Microservices Observability



In most cases, observability depends on three key types of insights that – when analyzed in a collective, correlative way – provide actionable insight into the state of complex microservices apps:

Logs

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

Microservices Observability: A Guide to Distributed System Visibility

Microservices Logging: Best Practices for Granular Insights into Distributed Systems

Microservices Monitoring: How to Choose the Right Tool

What is a Microservices Architecture? Key Characteristics, Use Cases & Benefits

Microservices Visualization: The Secret Ingredient in Effective Microservices Management

Products & Solutions ▾ Resources ▾ Company ▾ Customers ▾ Pricing ▾ Docs

● Example H2

Subscribe for updates

Email*

Subscribe

provide information on what each service instance does while running. If you want to know when an application error occurred or what the interval was between when a microservice accepted a request and when it finished processing it, for example, you'll typically find that insight in log files.

Metrics

Metrics record data such as the CPU and memory utilization of each microservice and the latency of requests. By tracking metrics over time, admins can identify anomalies, such as an unusual spike in CPU utilization, that could signal a problem. They can also correlate changes in metrics patterns with other data sources, like events recorded in log files, to gain deeper understanding of what caused a performance issue.

Traces

Traces provide feedback about how a request is processed as it flows through a distributed system. In other words, you can use **distributed tracing** to track how each microservice responds to a request.

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

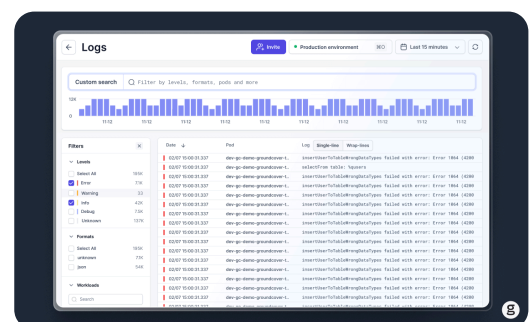
Accept

of a request processing issue. For instance, if you notice that a certain type of request takes your app a long time to process, you can run distributed tracing to determine which microservice is taking longest to handle its part of responding to the request.

Microservices Observability Patterns

The overarching goal of observability for microservices is to understand the internal state and health of complex, distributed applications. However, the act of observing microservices apps can be broken down into a variety of specific tasks, which are sometimes called patterns.

Logging Pattern



Microservices logging can be implemented in different ways, but the most common approach is

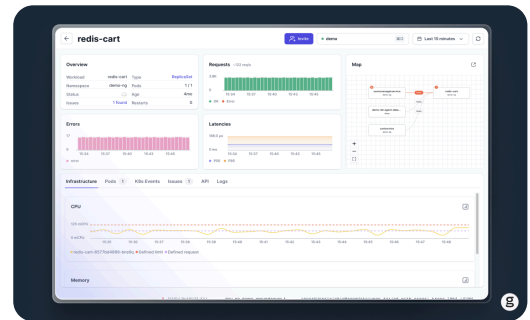
Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

collects logs from each microservice and moves log data into a tool that analyzes it.

This approach makes it easy to correlate and compare log files from each microservice to understand how an event recorded by one microservice relates to events generated by others.

Application Metrics Pattern



Microservice metrics collection involves tracking metrics associated with each microservice, as well as metrics for the entire application hosting stack. Based on this data, it becomes possible to identify where metrics anomalies occur and analyze their scope.

For example, if you detect a CPU usage spike in one microservice, having metrics data from other microservices will allow you to determine whether any other

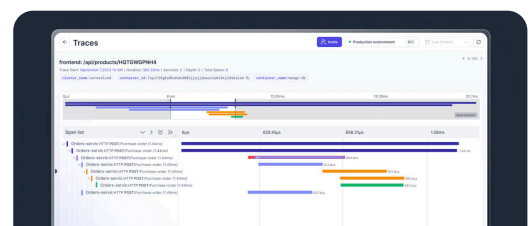
Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

you can check infrastructure metrics, such as how much CPU is being used by the node that hosts the microservice experiencing a spike, in order to decide whether you're at risk of running out of sufficient resources to keep the microservice running smoothly.

Methods for collecting microservices application metrics vary depending on exactly how the application is designed and how it is deployed; for example, tracking metrics for microservices running across a cluster of servers requires a different approach from metrics monitoring for microservices hosted on a single server. In general, however, you can collect metrics data at the operating system level. Orchestration tools may also offer some utilities for metrics collection; for instance, Kubernetes provides a metrics API as part of the built-in **Kubernetes observability** tooling.

Distributed Tracing Pattern



Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

Unlike microservices logs and metrics, distributed tracing for microservices isn't necessarily something that admins need to monitor on a continuous basis. Instead, they typically run traces when they detect a problem related to application requests and want more insight into it.

For example, you might notice from log data that some user requests are resulting in errors. But just because one microservice logged the errors doesn't necessarily mean that that microservice is the root cause of the error; it's possible that another microservice is doing something wrong that is triggering an error event. By running a distributed trace, you can monitor how each microservice responds to a request, allowing you to pinpoint the source of the error.

Exception Tracking

In some cases, your microservices may experience exceptions. Exceptions are similar to errors, but the key difference is that exceptions usually originate from problems with application code, whereas errors result from environment issues like lack of

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

Like errors, exceptions are a type of event that admins typically will want to investigate by running traces, which will identify the microservice at the root of the problem. From there, they can work with developers to debug the microservice, determine which code is triggering the issue, fix it and redeploy an updated version of the microservice.

Health Check API

In most cases, microservices rely on APIs to communicate with each other. In addition, the microservices app as a whole may receive requests via APIs that allow other applications or users to interact with it.

For these reasons, the ability to implement a service method for running health checks on all of the APIs at stake in a microservices app is critical. API health check service methods provide insights like how long it takes to respond to an API request or whether any types of requests result in errors or malformed responses. Admins will want to get ahead of API issues like these before they cause large-scale performance issues for your application.

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

Auditing

Auditing helps admins ensure that software usage patterns conform with any rules or standards that the business needs to follow. For example, compliance regulations may require that certain types of data be encrypted.

Auditing is a complex process that typically involves more than just observability, but **observability tools** and procedures can help handle some of this work. For instance, if developers build logic for generating audit logs into microservices, each audit log entry can be collected alongside event logs data, then analyzed to detect risky behavior. Likewise, changes to application state can be tracked in audit logs to ensure compliance with mandates such as specific uptime guarantees.

Observability Challenges for Microservices

Challenge	Why it's a problem
Complex	Distributed

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)



Challenge	Why it's a problem
Multiple potential causes of failure	Surface-level data about a problem doesn't always reveal the actual nature of the issue.
Data volume	Microservices produce large volumes of observability data, creating more information to analyze and correlate.
Inconsistencies	Microservices may be built and deployed in a variety of ways, and observability strategies must be able to adapt to all of them.

Observing any type of application can be challenging. But observability tends to be especially challenging when you're working with microservices, due to several factors:

Microservices Architecture

The greatest challenge is that microservices architectures are

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)



application from a single place. But with microservices, you need to track the behavior of a variety of different services. The fact that it's common to host microservices across a cluster of servers complicates matters further because it means that data sources from multiple services are spread across multiple nodes.

On top of this, cloud-native hosting stacks often involve multiple layers and complex abstractions. A common strategy is to deploy each service instance in a container running on top of a virtual machine, which is in turn part of a physical server. The containers are orchestrated using Kubernetes, and there may be load balancers, service meshes and other networking tools in the mix. All of this complexity translates to a huge amount of data and variables to sort through when observing the state of a microservices app.

Identifying Underlying Causes of Distributed Systems Failure

There are a variety of potential causes of failure in a system, including:

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

- Application exceptions triggered by buggy code.
- Lack of resources, such as insufficient CPU or memory, due to undersized infrastructure or "noisy neighbors" (meaning workloads that consume excess resources, leaving inadequate resources for other workloads to use).
- Misconfigured environment layers, such as incorrect settings in Kubernetes that cause a microservice to run on a node that lacks resources to host it.

Figuring out which type of failure is at play when something goes wrong can be challenging because it's not always clear from surface-level data alone what the nature of an error is. For instance, a spike in service request latency could be due to a variety of factors – buggy application code, insufficient resources or a misconfigured load balancer, to name just a few potential causes. Identifying the specific type of error you're facing requires the ability to correlate multiple data sources.

On top of this, you face (as we

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

is at the root cause of an error. In other words, simply knowing which type of error you're dealing with is only half the battle in observability for microservices; you also must determine which microservice is triggering the error.

Data Volume

When you're observing microservices, not only do you have more individual log files, metrics and other data sources to work with, but the overall volume of your data is also likely to be dramatically larger. As a result, analyzing the data to identify relevant insights is more challenging because there is more information to process and correlate.

Inconsistent Environments and Microservices

Microservices is a high-level concept that can be implemented in a variety of ways. This means that observability strategies that work for one microservices app – or even for one specific microservice – may not work for others due to differences like the programming language used to

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

As a result, observability for microservices strategies must be especially flexible and adaptable.

Best Practices for Implementing Observability for Microservices

To get the most out of observability for microservices, consider the following best practices:

- **Correlate, correlate, correlate:** We said it above but we'll say it again: Correlating different data sources is the key to observability for microservices. It's only by analyzing logs, metrics and traces collectively that you can gain real insights into complex performance issues.
- **Centralize data collection:** You can make the observability for microservices process faster and more efficient by using a centralized service to collect relevant data from across your environment – including not just from microservices themselves, but also from the

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

- **Standardize observability data:** To the extent possible, ensure that each service instance generates logs, metrics and distributed tracing data in a consistent way. This makes it easier to correlate and compare data across microservices.
- **Track observability overhead:** Running the tools and services that enable observability places extra load on your environments. While some level of overhead in this respect is unavoidable, it's important to track how many additional CPU, memory and other resources your observability solutions consume so that you don't end up depriving your actual workloads of the resources they require to function properly.

Microservices Observability Tools

Most major observability solutions available today can support microservices apps well. However, the approach that they take to collecting and processing observability data varies with

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

Observability for microservices tools can be categorized as follows:

- **Agent-based tools:** Some observability tools run traditional, user-space software agents to collect observability data from microservices. They typically do this by deploying agents either directly on host servers or (in environments like Kubernetes) as sidecar containers. This approach provides the ability to collect extensive data, but it also leads to significant resource overhead due to the resources consumed by the agents.
- **Agentless observability:** An agentless observability approach collects data via external tools and services. The advantage of this approach is that there is less overhead. However, you also get less visibility because the types of data you can collect using external services is limited.
- **eBPF-based observability:** A new generation of observability tools, including groundcover, leverage a framework called eBPF to

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

hosts microservices. Unlike traditional software agents, eBPF collectors run in kernel space, which makes them much more efficient. At the same time, they have complete access to all relevant data, so they are not subject to the visibility limitations of agentless observability.

Keep your microservices applications running smoothly and reliably

Observability for microservices is challenging. It requires evolving application performance management strategies to include more than mere monitoring. In addition, observability for microservices requires the ability to collect and correlate data from a wide variety of discrete sources, at

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

The good news is that, given the right tools and strategies, you can conquer these challenges and optimize the performance of microservices apps. When you update your observability strategy for the modern, cloud-native world, understanding the state of even the most complex distributed environment becomes possible.



Aviv Zohari, Founding Engineer
6 minutes read, May 30th, 2023

 **Copy link**

Sign up for Updates

Keep up with all things cloud-native observability.



Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept

We care about data. Check out our [privacy policy](#).

groundcover

groundcover is a cloud-native observability platform powered by eBPF that breaks free from the limits of SaaS. Built for modern production environments, it runs entirely in your cloud to deliver full visibility at a fraction of the cost, with no blind spots, no compromises, and no operational overhead.



Product

Infrastructure Monitoring
Application Performance Monitoring
Log Management
Real User Monitoring
Migration
eBPF Sensor
BYOC Architecture
Integrations
groundcover MCP
LLM Observability

[Launch](#)

[Playground](#)

Solutions

AWS Monitoring
Google Cloud Monitoring
Azure Monitoring
Kubernetes monitoring
Cost Efficiency
Replace OSS Stack
On Premise Solutions

Resources

Blog
Customer Stories
Events & Webinars
eBPF
OpenTelemetry
Kubernetes Monitoring
Kubernetes Troubleshooting
Microservices Observability
Observability Tool Comparison
Documentation

Company

About Us
Brand Guidelines
Contact Us
Partners
Careers
Pricing
In the News



[Terms of Use](#)

[Privacy Policy](#)

[Trust Center](#)

[HaMasger 47, Tel Aviv](#)

Info@groundcover.com

Yeah, somehow cookie consent is still a thing... accept them and move on or see what it all means [here](#)

Accept